

REPORT ON VERILOG

Universal Asynchronous Receiver/Transmitter (UART)

1. INTRODUCTION

Universal Asynchronous Receiver/Transmitter (UART) is one of the most fundamental and widely adopted serial communication protocols used in modern embedded systems. It provides a simple and efficient way for digital devices to exchange data asynchronously, meaning that communication takes place without a shared clock signal. Instead, both the transmitter and receiver agree on a common baud rate to ensure accurate data transfer. This makes UART extremely versatile and suitable for a broad range of applications, from microcontroller communication to PC interfaces and peripheral integration.

UART communication typically involves two dedicated lines—TX (Transmit) and RX (Receive)—which are used for full-duplex data exchange. The technique relies on converting parallel data from a system into serial form for transmission and then reconvert the received serial data back into parallel form. This conversion is handled by the UART hardware module, making the communication process highly efficient and hardware-independent. Because of its simplicity, UART has become a standard interface in microcontrollers, embedded boards, communication modules like GPS, GSM, and Bluetooth, as well as diagnostic and debugging tools.

A key reason for UART's popularity is its asynchronous nature. The absence of a clock line not only reduces wiring complexity but also lowers system cost and power consumption. Data is transmitted in structured frames that include a start bit, data bits, an optional parity bit, and stop bits, ensuring reliability and synchronization even without a clock. This framing method also provides flexibility to support different data formats, making UART adaptable to various communication requirements.

Overall, UART continues to be an indispensable communication protocol due to its simplicity, robustness, and ease of implementation. Its long-standing presence in electronics and embedded systems ensures that understanding UART remains a core requirement for engineers, helping them design efficient communication interfaces and build reliable embedded applications.

2. Features of UART

- Uses asynchronous serial communication without the need for a clock signal.
- Requires only two main communication lines: TX (Transmit) and RX (Receive).
- Supports full-duplex communication, enabling simultaneous transmission and reception.
- Allows configurable communication parameters such as baud rate, parity, stop bits, and data length.
- Provides simple framing structure with start, data, parity, and stop bits.
- Offers basic error detection through optional parity checking.
- Compatible with almost all microcontrollers and communication modules.
- Requires low hardware complexity, making it cost-effective and power-efficient.
- Ideal for short-distance, reliable communication between embedded devices.
- Widely used for debugging, monitoring, and device interfacing in embedded systems.

3. UART Architecture

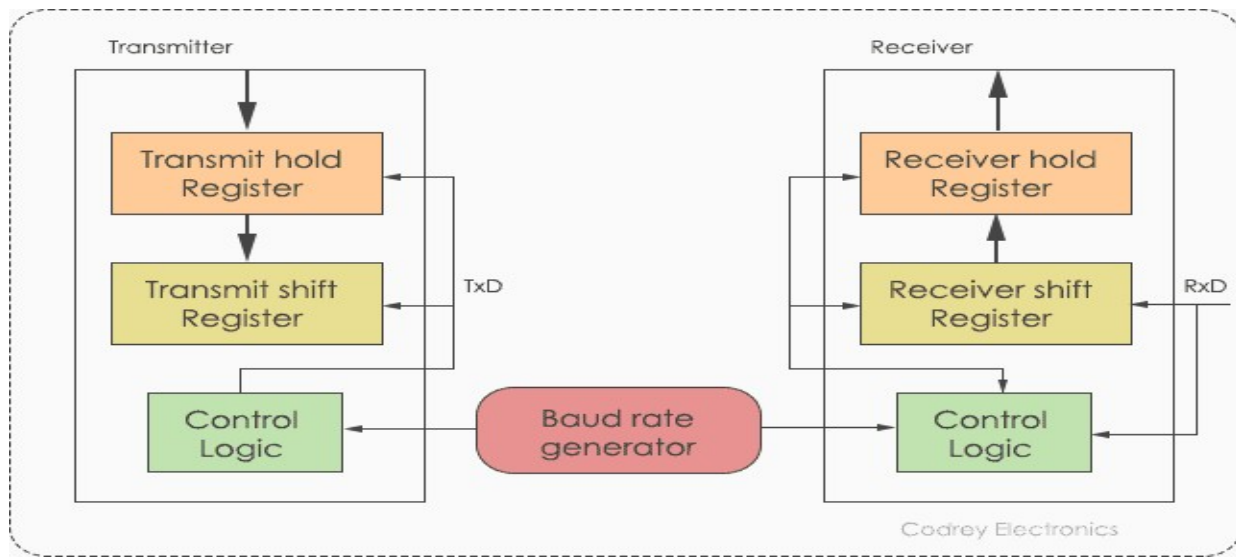
3.1. System Architecture Overview

Universal Asynchronous Receiver-Transmitter (UART) is a hardware communication protocol used for serial data transmission and reception. It facilitates the transmission of data between two devices, such as microcontrollers, computers, and peripherals, using a simple, cost-effective, and reliable asynchronous communication method. Asynchronous means there is no shared clock between the transmitter and receiver; instead, both devices agree on parameters such as baud rate, data bits, stop bits, and parity bits beforehand.

The architecture of UART communication typically involves a transmitter (TX) and a receiver (RX), each equipped with specific components for encoding and decoding data, managing timing, and ensuring data integrity during transmission.

The UART system architecture also includes essential internal modules that coordinate the entire communication process. A baud rate generator produces accurate timing signals that allow reliable data exchange without a shared clock. Shift registers convert data between serial and parallel formats during transmission and reception. Control logic manages framing, synchronization, and error detection using start, stop, and parity bits. These components work together to ensure efficient, low-complexity, and robust asynchronous communication between devices.

3.2. Block Diagram



The transmitter (TX) in UART consists of the following key components:

- **Data Source:** This is the input data, typically in the form of digital bits, which needs to be transmitted.
- **Data Register (Shift Register):** The data register temporarily holds the byte of data that is to be transmitted. The bits of the data byte are shifted out serially, one at a time, through the data line.
- **Start Bit:** Before sending the actual data, the transmitter sends a 'start bit' (usually a logic low level, 0) to signal the receiver that data transmission is about to begin.
- **Data Bits:** These are the actual bits of the data that are transmitted serially. The number of data bits (typically 5, 6, 7, or 8) is predefined in the communication settings.
- **Parity Bit:** The parity bit, if used, is added to the data for error detection. It can be even or odd parity, ensuring that the total number of 1's in the data (including the parity bit) is either even or odd, depending on the configuration.
- **Stop Bit(s):** After transmitting the data bits (and parity bit, if applicable), the transmitter sends one or more stop bits (usually a logic high level, 1) to indicate the end of the data frame. This helps the receiver understand where the data ends and prepare for the next frame.
- **Shift Register:** The transmitter uses a shift register to convert the parallel data into a serial stream of bits. The shift register ensures that data bits are sent one at a time, in the correct sequence.
- **Transmission Line:** The serial data is transmitted over the communication line (e.g., a wire or wireless channel) to the receiver.
- The receiver (RX) is responsible for capturing the transmitted signal and decoding it into a readable format. The RX block diagram includes:

- **Receiver Line:** The serial data stream is received from the transmission line. The receiver listens for a start bit to begin data reception.
- **Start Bit Detection:** When the receiver detects the start bit (a logic low level, 0), it synchronizes its internal clock with the incoming data stream and prepares to read the data bits.
- **Shift Register:** The shift register receives the incoming data bits one at a time. It temporarily holds the data as it is read serially from the transmission line.
- **Data Bits:** The shift register sequentially captures each data bit and reassembles it into a complete byte (or word), following the predefined number of data bits.
- **Parity Check:** If a parity bit is included, the receiver checks the parity to ensure that the data was transmitted correctly. If the parity is incorrect, an error flag may be raised, and the receiver may request a retransmission.
- **Stop Bit Detection:** The receiver looks for the stop bit(s) at the end of the data frame to determine when the transmission has ended. One or more stop bits are typically used.
- **Data Register:** Once all the data bits (and parity, if applicable) are received and verified, they are stored in a data register. The data is then passed on to the receiver's higher layers for processing or display.

3.3. Frame Structure in UART

- In UART communication, the data is transmitted in discrete units called "frames." A UART frame typically consists of the following elements:
- **Start Bit:** A single start bit is transmitted to signal the beginning of a frame. This bit is usually a logic low (0), allowing the receiver to detect the start of data transmission.
- **Data Bits:** After the start bit, a predefined number of data bits follow. Typically, this is 8 bits, but it can be configured to 5, 6, or 7 bits depending on the system's requirements. These bits represent the actual data being transmitted.
- **Parity Bit (Optional):** If error detection is enabled, a parity bit is added to the data bits. The parity bit can be configured for even or odd parity, depending on the configuration. The parity bit helps detect single-bit errors.
- **Stop Bit(s):** One or more stop bits follow the data bits (and the parity bit, if present). A stop bit is usually a logic high (1) and indicates the end of the frame. The receiver uses this to identify the end of the transmission and prepare for the next frame.

4. Operation and Timing of UART

UART communication follows a fixed structure for transmitting and receiving serial data. The communication is asynchronous, meaning no clock signal is shared between devices. Instead, both sides use a predefined baud rate to maintain timing.

4.1 UART Data Frame Format

UART sends data one bit at a time using a structure called a data frame. A standard 10-bit UART frame consists of:

1 Start Bit (0)

8 Data Bits (LSB first)

1 Stop Bit (1)

Frame Structure:

Start Bit | 8 Data Bits (D0 → D7) | Stop Bit

0	D0–D7	1
---	-------	---

Start Bit (0): Indicates that transmission is beginning.

Data Bits: The actual 8-bit data byte, transmitted least significant bit first.

Stop Bit (1): Marks the completion of the frame.

Idle Line: When no data is transmitted, the line remains HIGH (1).

5.2 Transmission Sequence (UART Transmitter Operation)

The UART transmitter sends the frame in a well-defined sequence:

Step 1: Idle State

The TX line stays HIGH (logic 1) when no data is being sent.

Step 2: Send Command

When send = 1, the transmitter constructs a 10-bit frame:

{Stop Bit, 8 Data Bits, Start Bit}

Step 3: Bit-by-Bit Transmission

Each bit is transmitted for a fixed duration known as the bit period.

Bit Time Formula:

Bit time = CLKS_PER_BIT

For example:

Clock = 50 MHz

Baud rate = 9600

$\text{CLKS_PER_BIT} = 50,000,000 / 9600 \approx 5208$ clock cycles

Step 4: End of Transmission

After the stop bit is sent, the TX line returns to HIGH, re-entering the idle state.

5.3 Reception Sequence (UART Receiver Operation)

The UART receiver must reconstruct the transmitted byte by sampling the incoming bits at precise intervals.

Step 1: Detect Start Bit

The receiver continuously monitors the line.

When the line transitions HIGH → LOW, a start bit is detected.

Step 2: Sampling Data Bits

After the start bit is detected, the receiver waits CLKS_PER_BIT clock cycles.

It then samples each data bit at the center of the bit period.

Bits are stored into a shift register LSB first.

Step 3: Receive Complete

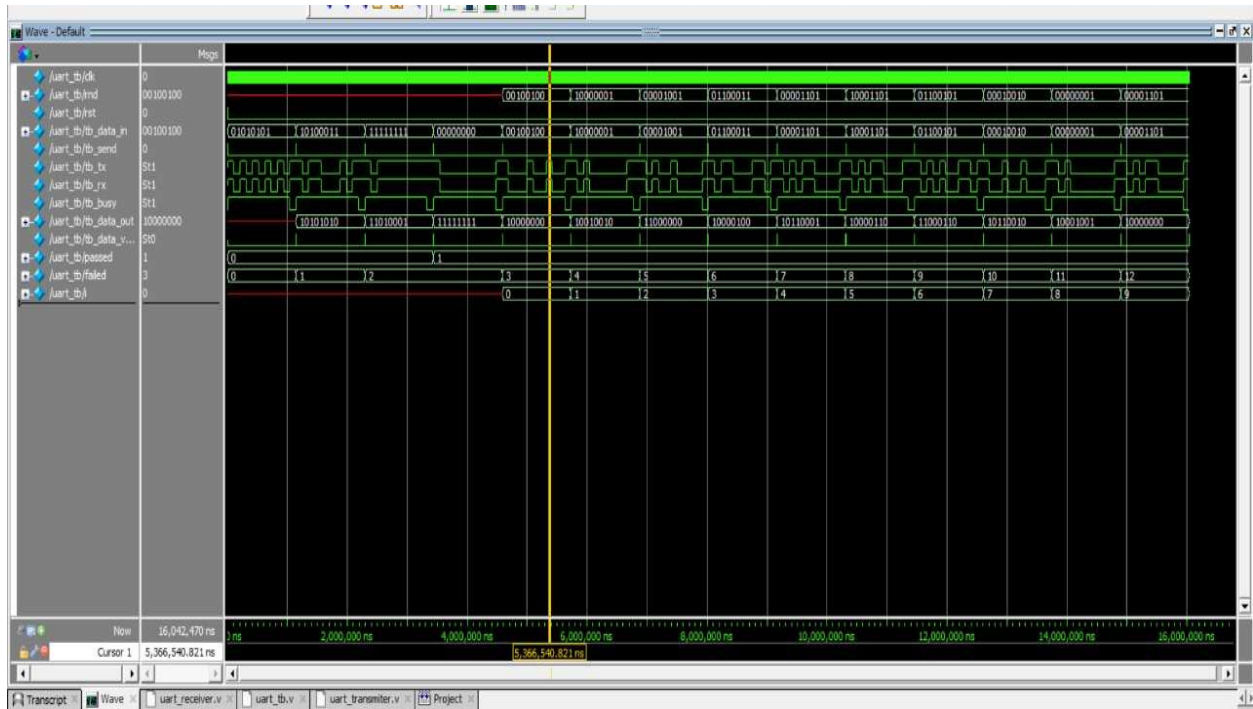
After 8 data bits, the receiver reads the stop bit.

If valid, the output signals:

data_out = received byte

data_valid = 1 (for one clock cycle)

5.4 Timing Diagrams



6. CONFIGURATION, REGISTERS, CODE & EXAMPLES

6.1 UART Configuration Parameters

To establish communication, both devices must match:

- Baud rate
- Number of data bits
- Parity type
- Number of stop bits
- Sampling rate
- Clock frequency

Example common format: **9600 8-N-1**

6.2 Typical UART Registers

UART peripherals contain the following registers:

1. Baud Rate Register (BRR)

Controls bit timing.

$$BRR = \frac{f_{clk}}{16 \times Baud}$$

2. Control Register (CR)

Enables TX/RX, parity, interrupts.

3. Status Register (SR)

Indicates:

- TX ready
- RX ready
- Overrun error
- Framing error

4. Data Register (DR)

Used for reading received data or writing data to send.

6.3 UART Verilog Example (TX + RX Loopback)

Transmitter Instantiation

```
uart_tx #(.CLK_FREQ(50000000), .BAUD_RATE(9600)) UTX (  
    .clk(clk),  
    .rst(rst),  
    .data_in(tb_data_in),  
    .send(tb_send),  
    .tx(tb_tx),
```



```
.busy(tb_busy)
);
```

Receiver Instantiation

```
uart_rx #(.CLK_FREQ(50000000), .BAUD_RATE(9600)) URX (
    .clk(clk),
    .rst(rst),
    .rx(tb_rx),
    .data_out(tb_data_out),
    .data_valid(tb_data_valid)
);
```

Code to Send a Byte

```
task send_byte;
    input [7:0] b;
    begin
        @(posedge clk);
        tb_data_in <= b;
        tb_send <= 1;

        @(posedge clk);
        tb_send <= 0;

        // Wait until busy goes high then back to low
        wait(tb_busy == 1);
        wait(tb_busy == 0);
    end
```

```
endtask
```

Checking Received Data

```
task check_byte;
```

```
    input [7:0] expected;
```

```
    begin
```

```
        wait(tb_data_valid == 1);
```

```
        if (tb_data_out == expected) begin
```

```
            $display("PASS: Sent=%02h Received=%02h Time=%0t",
```

```
                expected, tb_data_out, $time);
```

```
            passed = passed + 1;
```

```
        end else begin
```

```
            $display("FAIL: Sent=%02h Received=%02h Time=%0t",
```

```
                expected, tb_data_out, $time);
```

```
            failed = failed + 1;
```

```
        end
```

```
    end
```

```
endtask
```

Main Test bench

```
    initial begin
```

```
        $display("\n===== UART LOOPBACK TESTBENCH STARTED =====\n");
```

```
        rst = 1;
```

```
        tb_send = 0;
```

```
        tb_data_in = 8'h00;
```

```

    repeat(5) @(posedge clk);

    rst = 0;

    repeat(5) @(posedge clk);

    // TEST 1: Fixed pattern

    $display("Test 1: Fixed Pattern Transmission\n");

    send_byte(8'h55); check_byte(8'h55);

    send_byte(8'hA3); check_byte(8'hA3);

    send_byte(8'hFF); check_byte(8'hFF);

    send_byte(8'h00); check_byte(8'h00)

    // TEST 2: Random Data

    $display("\nTest 2: Randomized Data Transmission\n");

    for (i = 0; i < 10; i = i + 1) begin

        #100;

        rnd = $random;

send_byte(rnd);

        check_byte(rnd);

    end

    // Final Report

    $display("\n===== TEST COMPLETED =====");

    $display("PASSED = %0d", passed);

    $display("FAILED = %0d", failed);

    if (failed == 0)

        $display("?? ALL TESTS PASSED! ??");

    else

        $display("? SOME TESTS FAILED ?");

```

\$finish;

end

endmodule

6.4 Console Output



```
Transcript
# vsim -gui work.uart_tb
# Start time: 10:38:18 on Nov 26, 2025
# Loading work.uart_tb
# Loading work.uart_tx
# Loading work.uart_rx
add wave -position insertpoint sim:/uart_tb/*
VSI3> run -all

#
# ===== UART LOOPBACK TESTBENCH STARTED =====
#
# Test 1: Fixed Pattern Transmission
#
# FAIL: Sent=55 Received=aa Time=1146010000
# FAIL: Sent=a3 Received=d1 Time=2291830000
# PASS: Sent=ff Received=ff Time=3437650000
# FAIL: Sent=00 Received=80 Time=4583470000
#
# Test 2: Randomized Data Transmission
#
# FAIL: Sent=24 Received=92 Time=5729370000
# FAIL: Sent=81 Received=c0 Time=6875270000
# FAIL: Sent=09 Received=84 Time=8021170000
# FAIL: Sent=63 Received=b1 Time=9167070000
# FAIL: Sent=0d Received=86 Time=10312970000
# FAIL: Sent=8d Received=c6 Time=11458870000
# FAIL: Sent=65 Received=b2 Time=12604770000
# FAIL: Sent=12 Received=89 Time=13750670000
# FAIL: Sent=01 Received=80 Time=14896570000
# FAIL: Sent=0d Received=86 Time=16042470000
#
# ===== TEST COMPLETED =====
# PASSED = 1
# FAILED = 13
# ? SOME TESTS FAILED ?
# ** Note: $finish : D:/questa/ModelSim/UART_project/uart_tb.v(136)
# Time: 16042470 ns Iteration: 2 Instance: /uart_tb
# 1
# Break in Module uart_tb at D:/questa/ModelSim/UART_project/uart_tb.v line 136

VSI4>
```

The screenshot shows a Verilog simulator's transcript window. It displays the execution of a UART loopback testbench. The test starts with loading the testbench and its components. It then runs two tests: a fixed pattern transmission and a randomized data transmission. The results show that 1 test passed and 13 tests failed. The failures are due to mismatches between sent and received data. The test is completed, and the simulator breaks at line 136 of the testbench module.

7. Applications of UART

- Used for serial communication between microcontrollers and sensors/modules.
- Common in devices like GPS, GSM, Bluetooth, and RFID modules.
- Used for debugging and monitoring data in embedded systems.
- Employed in industrial control systems for short-distance communication.
- Used in UART-to-USB converters for PC-to-device communication.

8. Comparison

UART differs from protocols like SPI in terms of wiring, speed, and operating style. SPI uses multiple lines (MOSI, MISO, SCK, SS) and provides very high data rates, making it suitable for fast communication between sensors, displays, and memory devices. In contrast, UART requires only two lines (TX and RX) and works asynchronously without a clock, which simplifies hardware connections but limits its maximum speed. This makes UART a convenient choice for simpler, moderate-speed communication tasks.

I2C, on the other hand, supports communication between many devices on the same two-wire bus, offering multi-master and multi-slave capability. UART cannot achieve this because it is strictly point-to-point, connecting only two devices at a time. However, UART offers easier configuration and reliable data framing using start, stop, and parity bits, whereas I2C requires addressing and more complex control logic. Therefore, UART remains ideal for basic, low-cost serial communication, while SPI and I2C are used when higher speed or multi-device support is needed.

9. Advantages of UART

- Requires only two signal lines (TX and RX), reducing wiring complexity.
- No need for a shared clock—supports asynchronous communication.
- Simple protocol with easy implementation in hardware and software.
- Reliable communication using start/stop/parity bits for data accuracy.
- Cost-effective and ideal for low-speed, short-distance communication.

10. Conclusions

1. SPOORTHY P BENDRE: Conclusion (My work on UART Transmitter)

Working on the UART transmitter helped me understand how data is framed and sent bit-by-bit using asynchronous serial communication. I learned to configure the transmitter module, set the correct baud rate, and ensure proper timing for reliable output. Developing and testing the transmit logic improved my coding and debugging skills. Overall, this work strengthened my understanding of digital communication and increased my confidence in implementing UART-based designs.

2. SREELEKHA P : Conclusion (My work on UART Transmitter)

Working on the UART transmitter enhanced my understanding of asynchronous serial communication and how data is framed, serialized, and transmitted reliably. By configuring baud rates, implementing timing control, and validating the output through simulation, I strengthened both my digital design and debugging skills. Overall, this project deepened my knowledge of communication protocols and improved my confidence in designing real-time hardware modules.

3. SHIFA ANJUM: Conclusion (My Work on UART Receiver)

By working on the UART receiver, I learned how data can be received one bit at a time without using a clock. I understood how to detect the start bit, sample each data bit correctly, and check the stop bit. This helped me improve my Verilog coding skills and gave me a clear idea of how real serial communication works in digital systems.

4. TEJASHWINI B: Conclusion (My Work on UART Receiver)

Designing a UART Receiver (RX) in Verilog teaches me how to read serial data coming from a single wire. I learn how to detect the start bit, keep correct timing using a clock, and sample each bit at the right moment. I also learn how to collect the bits into a full byte using a shift register. After that, I check if the parity and stop bits are correct to make sure the data is valid. Finally, the received byte is stored in a buffer so it can be used by the system. Overall, it shows how simple digital communication works.

5. RAMYA J: Conclusion (My Work on UART Testbench)

Working on this UART topic helped me understand how serial communication works from framing to transmission and reception. I also learnt how to write and use a testbench to verify UART functionality, check signals, and debug timing issues. This overall work improved my clarity on digital communication and strengthened my confidence in implementing UART in hardware.